```
import pandas as pd
from matplotlib import pyplot as plt
import numpy as np
```

Load the Data set

```
df = pd.read_csv("C:\\Users\\ASUS\\OneDrive\\Desktop\\Data Science Project\\Customer-Churn.csv")
df.head()
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	...	Dev
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	DSL	No	...	
1	5575-GNVDE	Male	0	No	No	34	Yes	No	DSL	Yes	...	
2	3668-QPYBK	Male	0	No	No	2	Yes	No	DSL	Yes	...	
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	DSL	Yes	...	
4	9237-HQITU	Female	0	No	No	2	Yes	No	Fiber optic	No	...	

5 rows × 21 columns

Drop the Customer ID column

```
df.drop('customerID',axis='columns',inplace=True)
```

Remove the Duplicates

```
df.drop_duplicates()
```

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	OnlineBackup	Dev
0	Female	0	Yes	No	1	No	No phone service	DSL	No	Yes	
1	Male	0	No	No	34	Yes	No	DSL	Yes	No	
2	Male	0	No	No	2	Yes	No	DSL	Yes	Yes	
3	Male	0	No	No	45	No	No phone service	DSL	Yes	No	
4	Female	0	No	No	2	Yes	No	Fiber optic	No	No	
...	...	...	...	...	...	...	...	...	...	...	
7038	Male	0	Yes	Yes	24	Yes	Yes	DSL	Yes	No	
7039	Female	0	Yes	Yes	72	Yes	Yes	Fiber optic	No	Yes	
7040	Female	0	Yes	Yes	11	No	No phone service	DSL	Yes	No	
7041	Male	1	Yes	No	4	Yes	Yes	Fiber optic	No	No	
7042	Male	0	No	No	66	Yes	No	Fiber optic	Yes	No	

7021 rows × 20 columns

```
df.isnull().sum()
```

```
gender              0
SeniorCitizen       0
Partner             0
```

```
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges        0
Churn               0
dtype: int64
```

```
df.dtypes
```

```
gender               object
SeniorCitizen         int64
Partner              object
Dependents           object
tenure                int64
PhoneService         object
MultipleLines        object
InternetService      object
OnlineSecurity       object
OnlineBackup         object
DeviceProtection     object
TechSupport          object
StreamingTV          object
StreamingMovies      object
Contract             object
PaperlessBilling     object
PaymentMethod        object
MonthlyCharges      float64
TotalCharges         object
Churn                object
dtype: object
```

```
df.TotalCharges.values
```

```
array(['29.85', '1889.5', '108.15', ..., '346.45', '306.6', '6844.5'],
      dtype=object)
```

```
pd.to_numeric(df.TotalCharges)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
File lib.pyx:2391, in pandas._libs.lib.maybe_convert_numeric()

ValueError: Unable to parse string " "

During handling of the above exception, another exception occurred:

ValueError                                Traceback (most recent call last)
Cell In[8], line 1
----> 1 pd.to_numeric(df.TotalCharges)

File c:\Users\ASUS\AppData\Local\Programs\Python\Python312\Lib\site-packages\pandas\core\tools\numeric.py:232, in to_numeric(arg,
errors, downcast, dtype_backend)
    230 coerce_numeric = errors not in ("ignore", "raise")
    231 try:
--> 232     values, new_mask = lib.maybe_convert_numeric(  # type: ignore[call-overload]
    233         values,
    234         set(),
    235         coerce_numeric=coerce_numeric,
    236         convert_to_masked_nullable=dtype_backend is not lib.no_default
    237         or isinstance(values_dtype, StringDtype)
    238         and not values_dtype.storage == "pyarrow_numpy",
    239     )
    240 except (ValueError, TypeError):
    241     if errors == "raise":

File lib.pyx:2433, in pandas._libs.lib.maybe_convert_numeric()

ValueError: Unable to parse string " " at position 488
```

```
df['TotalCharges'] = pd.to_numeric(df.TotalCharges, errors='coerce')
df1 = df.dropna(subset=['TotalCharges'])
```

```
df1['Churn'].value_counts()
```

```
Churn
0    5163
1    1869
Name: count, dtype: int64
```

```
df1.shape
```

```
(7032, 20)
```

Change the data type of TotalCharges

```
df1.loc[:, 'TotalCharges'] = pd.to_numeric(df1.TotalCharges)
```

```
df1.TotalCharges.dtypes
```

```
dtype('float64')
```

Get the unique values in each columns

```
def print_unique_col_values(df):
       for column in df:
            if df[column].dtypes=='object':
                print(f'{column}: {df[column].unique()}')
```

```
print_unique_col_values(df1)
```

```
gender: ['Female' 'Male']
Partner: ['Yes' 'No']
Dependents: ['No' 'Yes']
PhoneService: ['No' 'Yes']
MultipleLines: ['No phone service' 'No' 'Yes']
InternetService: ['DSL' 'Fiber optic' 'No']
OnlineSecurity: ['No' 'Yes' 'No internet service']
OnlineBackup: ['Yes' 'No' 'No internet service']
DeviceProtection: ['No' 'Yes' 'No internet service']
TechSupport: ['No' 'Yes' 'No internet service']
StreamingTV: ['No' 'Yes' 'No internet service']
StreamingMovies: ['No' 'Yes' 'No internet service']
Contract: ['Month-to-month' 'One year' 'Two year']
PaperlessBilling: ['Yes' 'No']
PaymentMethod: ['Electronic check' 'Mailed check' 'Bank transfer (automatic)'
 'Credit card (automatic)']
Churn: ['No' 'Yes']
```

```
df1 = df1.copy()

df1.replace('No internet service','No',inplace=True)
df1.replace('No phone service','No',inplace=True)
```

```
print_unique_col_values(df1)
```

```
gender: ['Female' 'Male']
Partner: ['Yes' 'No']
Dependents: ['No' 'Yes']
PhoneService: ['No' 'Yes']
MultipleLines: ['No' 'Yes']
InternetService: ['DSL' 'Fiber optic' 'No']
OnlineSecurity: ['No' 'Yes']
OnlineBackup: ['Yes' 'No']
DeviceProtection: ['No' 'Yes']
TechSupport: ['No' 'Yes']
StreamingTV: ['No' 'Yes']
StreamingMovies: ['No' 'Yes']
Contract: ['Month-to-month' 'One year' 'Two year']
PaperlessBilling: ['Yes' 'No']
PaymentMethod: ['Electronic check' 'Mailed check' 'Bank transfer (automatic)'
 'Credit card (automatic)']
```

```
Churn: ['No' 'Yes']
```

Encoding

```
yes_no_columns = ['Partner','Dependents','PhoneService','MultipleLines','OnlineSecurity','OnlineBackup',
                  'DeviceProtection','TechSupport','StreamingTV','StreamingMovies','PaperlessBilling','Churn']
for col in yes_no_columns:
    df1[col].replace({'Yes': 1,'No': 0},inplace=True)
```

```
for col in df1:
    print(f'{col}: {df1[col].unique()}')
```

```
gender: ['Female' 'Male']
SeniorCitizen: [0 1]
Partner: [1 0]
Dependents: [0 1]
tenure: [ 1 34  2 45  8 22 10 28 62 13 16 58 49 25 69 52 71 21 12 30 47 72 17 27
  5 46 11 70 63 43 15 60 18 66  9  3 31 50 64 56  7 42 35 48 29 65 38 68
 32 55 37 36 41  6  4 33 67 23 57 61 14 20 53 40 59 24 44 19 54 51 26 39]
PhoneService: [0 1]
MultipleLines: [0 1]
InternetService: ['DSL' 'Fiber optic' 'No']
OnlineSecurity: [0 1]
OnlineBackup: [1 0]
DeviceProtection: [0 1]
TechSupport: [0 1]
StreamingTV: [0 1]
StreamingMovies: [0 1]
Contract: ['Month-to-month' 'One year' 'Two year']
PaperlessBilling: [1 0]
PaymentMethod: ['Electronic check' 'Mailed check' 'Bank transfer (automatic)'
 'Credit card (automatic)']
MonthlyCharges: [29.85 56.95 53.85 ... 63.1  44.2  78.7 ]
TotalCharges: [  29.85 1889.5   108.15 ...  346.45  306.6  6844.5 ]
Churn: [0 1]
```

```
df1['gender'].replace({'Female':1,'Male':0},inplace=True)
```

```
df1.gender.unique()
```

```
array([1, 0], dtype=int64)
```

Get the Dummy variables

```
df2 = pd.get_dummies(data=df1, columns=['InternetService','Contract','PaymentMethod'])
df2.columns
```

```
Index(['gender', 'SeniorCitizen', 'Partner', 'Dependents', 'tenure',
       'PhoneService', 'MultipleLines', 'OnlineSecurity', 'OnlineBackup',
       'DeviceProtection', 'TechSupport', 'StreamingTV', 'StreamingMovies',
       'PaperlessBilling', 'MonthlyCharges', 'TotalCharges', 'Churn',
       'InternetService_DSL', 'InternetService_Fiber optic',
       'InternetService_No', 'Contract_Month-to-month', 'Contract_One year',
       'Contract_Two year', 'PaymentMethod_Bank transfer (automatic)',
       'PaymentMethod_Credit card (automatic)',
       'PaymentMethod_Electronic check', 'PaymentMethod_Mailed check'],
      dtype='object')
```

```
df2.head()
```

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	OnlineSecurity	OnlineBackup	DeviceProtection	...
0	1	0	1	0	1	0	0	0	1	0	...
1	0	0	0	0	34	1	0	1	0	1	...
2	0	0	0	0	2	1	0	1	1	0	...
3	0	0	0	0	45	0	0	1	0	1	...
4	1	0	0	0	2	1	0	0	0	0	...

5 rows × 27 columns

```
df2.dtypes
```

```
gender                                    int64
SeniorCitizen                             int64
Partner                                   int64
Dependents                                int64
tenure                                    int64
PhoneService                              int64
MultipleLines                             int64
OnlineSecurity                            int64
OnlineBackup                              int64
DeviceProtection                          int64
TechSupport                               int64
StreamingTV                               int64
StreamingMovies                           int64
PaperlessBilling                          int64
MonthlyCharges                          float64
TotalCharges                            float64
Churn                                     int64
InternetService_DSL                        bool
InternetService_Fiber optic                bool
InternetService_No                         bool
Contract_Month-to-month                    bool
Contract_One year                          bool
Contract_Two year                          bool
PaymentMethod_Bank transfer (automatic)    bool
PaymentMethod_Credit card (automatic)      bool
PaymentMethod_Electronic check             bool
PaymentMethod_Mailed check                 bool
dtype: object
```

```
yes_no_columns_new = [
    "InternetService_DSL", "InternetService_Fiber optic", "InternetService_No",
    "Contract_Month-to-month", "Contract_One year", "Contract_Two year",
    "PaymentMethod_Bank transfer (automatic)", "PaymentMethod_Credit card (automatic)",
    "PaymentMethod_Electronic check", "PaymentMethod_Mailed check"
]

# Convert True to 1 and False to 0
for col in yes_no_columns_new:
    df2[col] = df2[col].astype(int)

df2.head(5)
```

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	OnlineSecurity	OnlineBackup	DeviceProtection	...
0	1	0	1	0	1	0	0	0	1	0	...
1	0	0	0	0	34	1	0	1	0	1	...
2	0	0	0	0	2	1	0	1	1	0	...
3	0	0	0	0	45	0	0	1	0	1	...
4	1	0	0	0	2	1	0	0	0	0	...

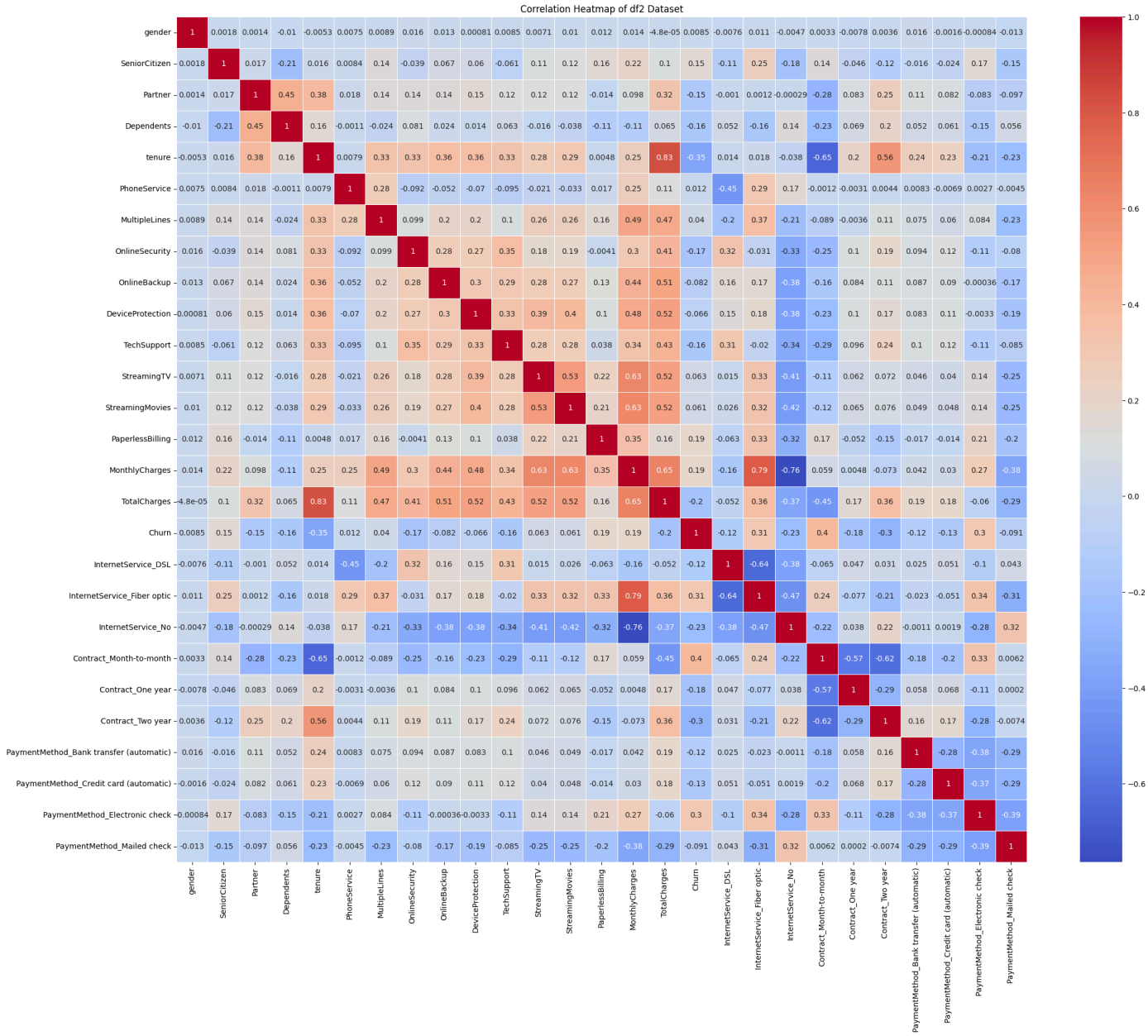
5 rows × 27 columns

```
import seaborn as sns
```

```
correlation_matrix = df2.corr()

plt.figure(figsize=(25, 20))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', linewidths=0.5)

plt.title('Correlation Heatmap of df2 Dataset')
plt.show()
```



Get the specific scaler

```
cols_to_scale = ['tenure','MonthlyCharges','TotalCharges']

from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
df2[cols_to_scale] = scaler.fit_transform(df2[cols_to_scale])
```

```
df2.head()
```

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	OnlineSecurity	OnlineBackup	DeviceProtection	...
0	1	0	1	0	0.000000	0	0	0	1	0	...
1	0	0	0	0	0.464789	1	0	1	0	1	...
2	0	0	0	0	0.014085	1	0	1	1	0	...
3	0	0	0	0	0.619718	0	0	1	0	1	...
4	1	0	0	0	0.014085	1	0	0	0	0	...

5 rows × 27 columns

```
for col in df2:
    print(f'{col}: {df2[col].unique()}')
```

```
gender: [1 0]
SeniorCitizen: [0 1]
Partner: [1 0]
Dependents: [0 1]
tenure: [0.         0.46478873 0.01408451 0.61971831 0.09859155 0.29577465
 0.12676056 0.38028169 0.85915493 0.16901408 0.21126761 0.8028169
 0.67605634 0.33802817 0.95774648 0.71830986 0.98591549 0.28169014
 0.15492958 0.4084507  0.64788732 1.         0.22535211 0.36619718
 0.05633803 0.63380282 0.14084507 0.97183099 0.87323944 0.5915493
 0.1971831  0.83098592 0.23943662 0.91549296 0.11267606 0.02816901
 0.42253521 0.69014085 0.88732394 0.77464789 0.08450704 0.57746479
 0.47887324 0.66197183 0.3943662  0.90140845 0.52112676 0.94366197
 0.43661972 0.76056338 0.50704225 0.49295775 0.56338028 0.07042254
 0.04225352 0.45070423 0.92957746 0.30985915 0.78873239 0.84507042
 0.18309859 0.26760563 0.73239437 0.54929577 0.81690141 0.32394366
 0.6056338  0.25352113 0.74647887 0.70422535 0.35211268 0.53521127]
PhoneService: [0 1]
MultipleLines: [0 1]
OnlineSecurity: [0 1]
OnlineBackup: [1 0]
DeviceProtection: [0 1]
TechSupport: [0 1]
StreamingTV: [0 1]
StreamingMovies: [0 1]
PaperlessBilling: [1 0]
MonthlyCharges: [0.11542289 0.38507463 0.35422886 ... 0.44626866 0.25820896 0.60149254]
TotalCharges: [0.0012751  0.21586661 0.01031041 ... 0.03780868 0.03321025 0.78764136]
Churn: [0 1]
InternetService_DSL: [1 0]
InternetService_Fiber optic: [0 1]
InternetService_No: [0 1]
Contract_Month-to-month: [1 0]
Contract_One year: [0 1]
Contract_Two year: [0 1]
PaymentMethod_Bank transfer (automatic): [0 1]
PaymentMethod_Credit card (automatic): [0 1]
PaymentMethod_Electronic check: [1 0]
PaymentMethod_Mailed check: [0 1]
```

Traing and Testing data set

```
X = df2.drop('Churn',axis='columns')
y = df2['Churn']

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.2,random_state=5)
```

ANN model

```
import tensorflow as tf
from tensorflow import keras


model = keras.Sequential([
    keras.Input(shape=(26,)),
    keras.layers.Dense(26, activation='relu'),
    keras.layers.Dense(15, activation='relu'),
    keras.layers.Dense(1, activation='sigmoid')
])

# opt = keras.optimizers.Adam(learning_rate=0.01)

model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])

model_train = model.fit(X_train, y_train, epochs=100)
```

```
Epoch 72/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8344 - loss: 0.3567
Epoch 73/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8385 - loss: 0.3564
Epoch 74/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8257 - loss: 0.3639
Epoch 75/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8378 - loss: 0.3522
Epoch 76/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8352 - loss: 0.3529
Epoch 77/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8365 - loss: 0.3556
Epoch 78/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8387 - loss: 0.3502
Epoch 79/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8281 - loss: 0.3618
Epoch 80/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8348 - loss: 0.3597
Epoch 81/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8377 - loss: 0.3459
Epoch 82/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8399 - loss: 0.3495
Epoch 83/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8381 - loss: 0.3513
Epoch 84/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8342 - loss: 0.3541
Epoch 85/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8270 - loss: 0.3642
Epoch 86/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8412 - loss: 0.3467
Epoch 87/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8362 - loss: 0.3533
Epoch 88/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8294 - loss: 0.3554
Epoch 89/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8324 - loss: 0.3526
Epoch 90/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8369 - loss: 0.3458
Epoch 91/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8383 - loss: 0.3439
Epoch 92/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8305 - loss: 0.3511
Epoch 93/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8347 - loss: 0.3582
Epoch 94/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8375 - loss: 0.3523
Epoch 95/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8310 - loss: 0.3647
Epoch 96/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8358 - loss: 0.3492
Epoch 97/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8368 - loss: 0.3507
Epoch 98/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8431 - loss: 0.3450
Epoch 99/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.8426 - loss: 0.3400
Epoch 100/100
176/176 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8453 - loss: 0.3351
```

Evaluate the model

```
model.evaluate(X_test, y_test)
```

```
44/44 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.7790 - loss: 0.4876
[0.4957716166973114, 0.7746979594230652]
```

```
yp = model.predict(X_test)
yp[:5]
```

```
44/44 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step
array([[0.34210768],
       [0.5630413 ],
       [0.01335127],
       [0.8159455 ],
       [0.6638485 ]], dtype=float32)
```

```
y_pred = []
for element in yp:
    if element > 0.5:
        y_pred.append(1)
    else:
        y_pred.append(0)
```

```
y_pred[:10]
```

```
[0, 1, 0, 1, 1, 1, 0, 0, 0, 0]
```

```
y_test[:10]
```

```
2660    0
744     0
5579    1
64      1
3287    1
816     1
2670    0
5920    0
1023    0
6087    0
Name: Churn, dtype: int64
```

```
from sklearn.metrics import classification_report

print(classification_report(y_test,y_pred))
```

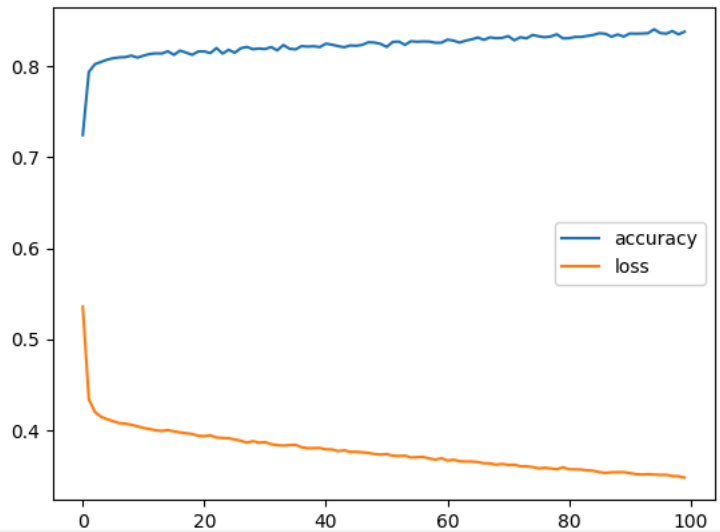
```
              precision    recall  f1-score   support

           0       0.84      0.84      0.84       999
           1       0.61      0.61      0.61       408

    accuracy                           0.77      1407
   macro avg       0.73      0.72      0.73      1407
weighted avg       0.77      0.77      0.77      1407
```

```
pd.DataFrame(model_train.history).plot()
```

```
<Axes: >
```



Fixed the imbalance Error

```
X = df2.drop('Churn', axis='columns')
y = df2['Churn']
```

```
from imblearn.over_sampling import SMOTE

smote = SMOTE(sampling_strategy='minority')
X_sm, y_sm = smote.fit_resample(X, y)

y_sm.value_counts()
```

```
Churn
0    5163
1    5163
Name: count, dtype: int64
```

```
from sklearn.model_selection import train_test_split
X_train_new, X_test_new, y_train_new, y_test_new = train_test_split(X_sm, y_sm, test_size=0.2, random_state=15, stratify= y_sm)
```

```
y_train_new.value_counts()
```

```
Churn
1    4130
0    4130
Name: count, dtype: int64
```

```
import tensorflow as tf
from tensorflow import keras

model = keras.Sequential([
    keras.Input(shape=(26,)),
    keras.layers.Dense(26, activation='relu'),
    keras.layers.Dense(15, activation='relu'),
    keras.layers.Dense(1, activation='sigmoid')
])

# opt = keras.optimizers.Adam(learning_rate=0.01)

model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])

model_train_new = model.fit(X_train_new, y_train_new, epochs=100)
```

```
259/259 ━━━━━━━━━━━━━━━━━━━━ 0s 673us/step - accuracy: 0.8391 - loss: 0.3598
Epoch 75/100
259/259 ━━━━━━━━━━━━━━━━━━━━ 0s 685us/step - accuracy: 0.8451 - loss: 0.3546
Epoch 76/100
259/259 ━━━━━━━━━━━━━━━━━━━━ 0s 677us/step - accuracy: 0.8469 - loss: 0.3518
Epoch 77/100
259/259 ━━━━━━━━━━━━━━━━━━━━ 0s 618us/step - accuracy: 0.8434 - loss: 0.3536
Epoch 78/100
259/259 ━━━━━━━━━━━━━━━━━━━━ 0s 679us/step - accuracy: 0.8425 - loss: 0.3598
Epoch 79/100
259/259 ━━━━━━━━━━━━━━━━━━━━ 0s 629us/step - accuracy: 0.8415 - loss: 0.3610
Epoch 80/100
259/259 ━━━━━━━━━━━━━━━━━━━━ 0s 655us/step - accuracy: 0.8514 - loss: 0.3452
Epoch 81/100
259/259 ━━━━━━━━━━━━━━━━━━━━ 0s 796us/step - accuracy: 0.8397 - loss: 0.3578
Epoch 82/100
259/259 ━━━━━━━━━━━━━━━━━━━━ 0s 646us/step - accuracy: 0.8398 - loss: 0.3584
Epoch 83/100
259/259 ━━━━━━━━━━━━━━━━━━━━ 0s 710us/step - accuracy: 0.8471 - loss: 0.3417
Epoch 84/100
259/259 ━━━━━━━━━━━━━━━━━━━━ 0s 673us/step - accuracy: 0.8433 - loss: 0.3553
Epoch 85/100
259/259 ━━━━━━━━━━━━━━━━━━━━ 0s 708us/step - accuracy: 0.8450 - loss: 0.3502
Epoch 86/100
259/259 ━━━━━━━━━━━━━━━━━━━━ 0s 659us/step - accuracy: 0.8459 - loss: 0.3440
Epoch 87/100
259/259 ━━━━━━━━━━━━━━━━━━━━ 0s 705us/step - accuracy: 0.8444 - loss: 0.3515
Epoch 88/100
259/259 ━━━━━━━━━━━━━━━━━━━━ 0s 647us/step - accuracy: 0.8398 - loss: 0.3540
Epoch 89/100
259/259 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.8384 - loss: 0.3584
Epoch 90/100
259/259 ━━━━━━━━━━━━━━━━━━━━ 0s 669us/step - accuracy: 0.8397 - loss: 0.3542
Epoch 91/100
259/259 ━━━━━━━━━━━━━━━━━━━━ 0s 663us/step - accuracy: 0.8360 - loss: 0.3612
Epoch 92/100
259/259 ━━━━━━━━━━━━━━━━━━━━ 0s 738us/step - accuracy: 0.8371 - loss: 0.3555
Epoch 93/100
259/259 ━━━━━━━━━━━━━━━━━━━━ 0s 818us/step - accuracy: 0.8498 - loss: 0.3549
Epoch 94/100
259/259 ━━━━━━━━━━━━━━━━━━━━ 0s 779us/step - accuracy: 0.8440 - loss: 0.3585
Epoch 95/100
259/259 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.8463 - loss: 0.3434
Epoch 96/100
259/259 ━━━━━━━━━━━━━━━━━━━━ 0s 821us/step - accuracy: 0.8513 - loss: 0.3375
Epoch 97/100
259/259 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.8430 - loss: 0.3516
Epoch 98/100
259/259 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.8525 - loss: 0.3383
Epoch 99/100
259/259 ━━━━━━━━━━━━━━━━━━━━ 0s 866us/step - accuracy: 0.8402 - loss: 0.3589
Epoch 100/100
259/259 ━━━━━━━━━━━━━━━━━━━━ 0s 825us/step - accuracy: 0.8438 - loss: 0.3485
```

```
model.evaluate(X_test_new, y_test_new)
```

```
65/65 ━━━━━━━━━━━━━━━━━━━━ 0s 647us/step - accuracy: 0.8102 - loss: 0.4171
[0.42072468996047974, 0.8102613687515259]
```

```
y_predict_new = model.predict(X_test_new)
```

```
65/65 ━━━━━━━━━━━━━━━━━━━━ 0s 851us/step
```

```
y_predict_new[:5]
```

```
array([[0.76604867],
       [0.00591876],
       [0.5708025 ],
       [0.8959086 ],
       [0.03090531]], dtype=float32)
```

```
y_pred_new = []
for element in y_predict_new:
    if element > 0.5:
        y_pred_new.append(1)
    else:
        y_pred_new.append(0)
```

```
y_pred_new[:10]
```

```
[1, 0, 1, 1, 0, 0, 0, 1, 1, 1]
```

```
y_test_new[:10]
```

```
3045     1
4723     0
10271    1
9289     1
5026     0
3950     0
1036     1
2993     0
9954     1
1079     1
Name: Churn, dtype: int64
```

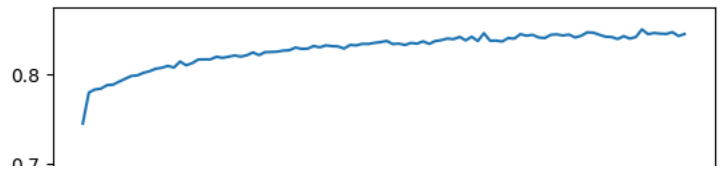
```
from sklearn.metrics import classification_report

print(classification_report(y_test_new,y_pred_new))
```

```
              precision    recall  f1-score   support

           0       0.82      0.80      0.81      1033
           1       0.80      0.82      0.81      1033

    accuracy                           0.81      2066
   macro avg       0.81      0.81      0.81      2066
weighted avg       0.81      0.81      0.81      2066
```

```
pd.DataFrame(model_train_new.history).plot()
```

```
<Axes: >
```